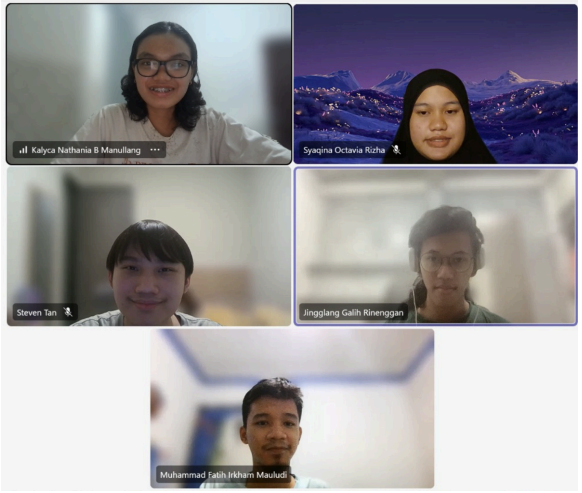
Laporan M2 Tugas Besar 1
IF2010 Pemrograman Berorientasi Objek 2025/2026
Nimonspoli

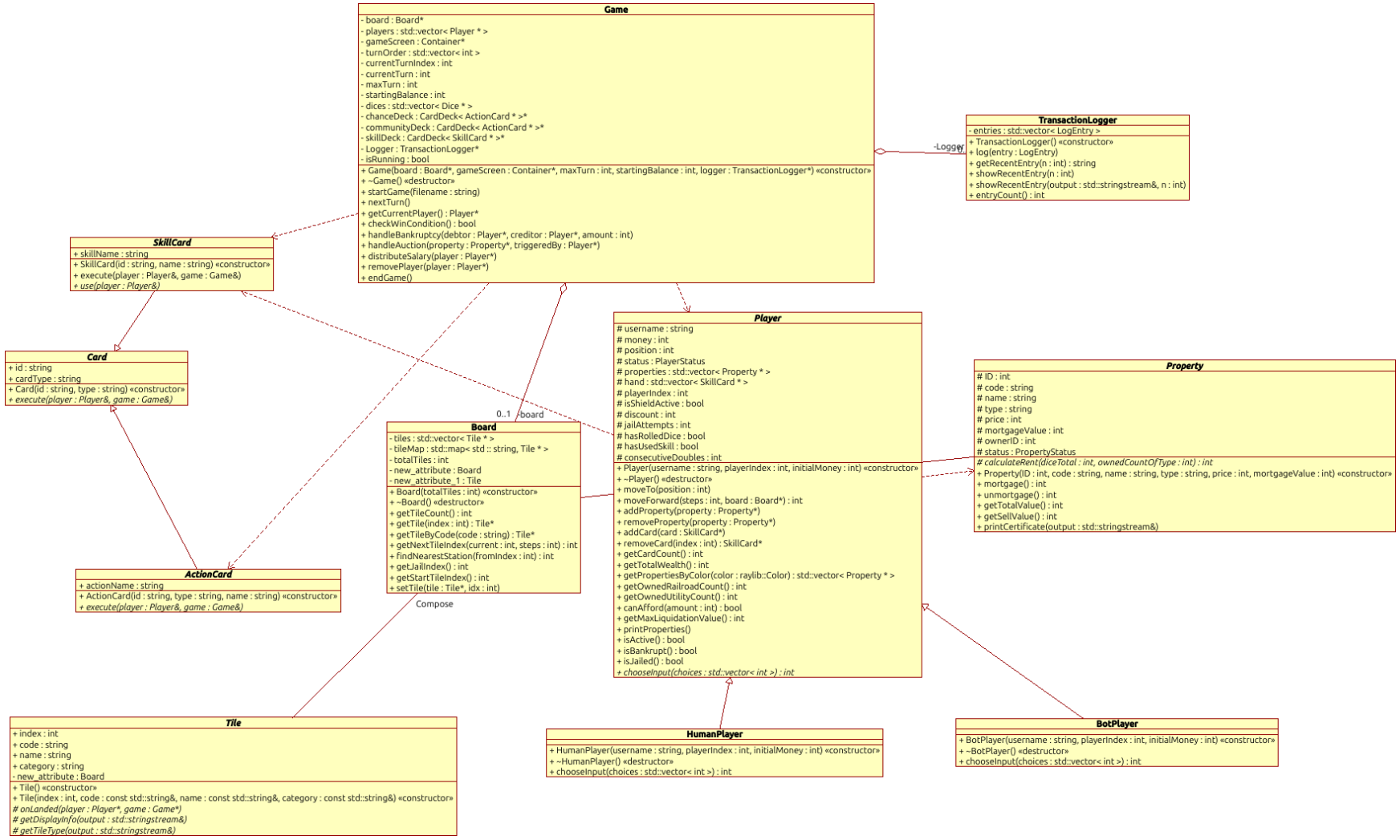
Kode Kelompok : SSR
Nama Kelompok : kata Fatih terserah sih

1. 13524004 / Muhammad Fatih Irkham Mauludi
2. 13524060 / Steven Tan
3. 13524071 / Kalyca Nathania Benedicta Manullang
4. 13524088 / Syaqina Octavia Rizha
5. 13524095 / Jingglang Galih Rinenggan

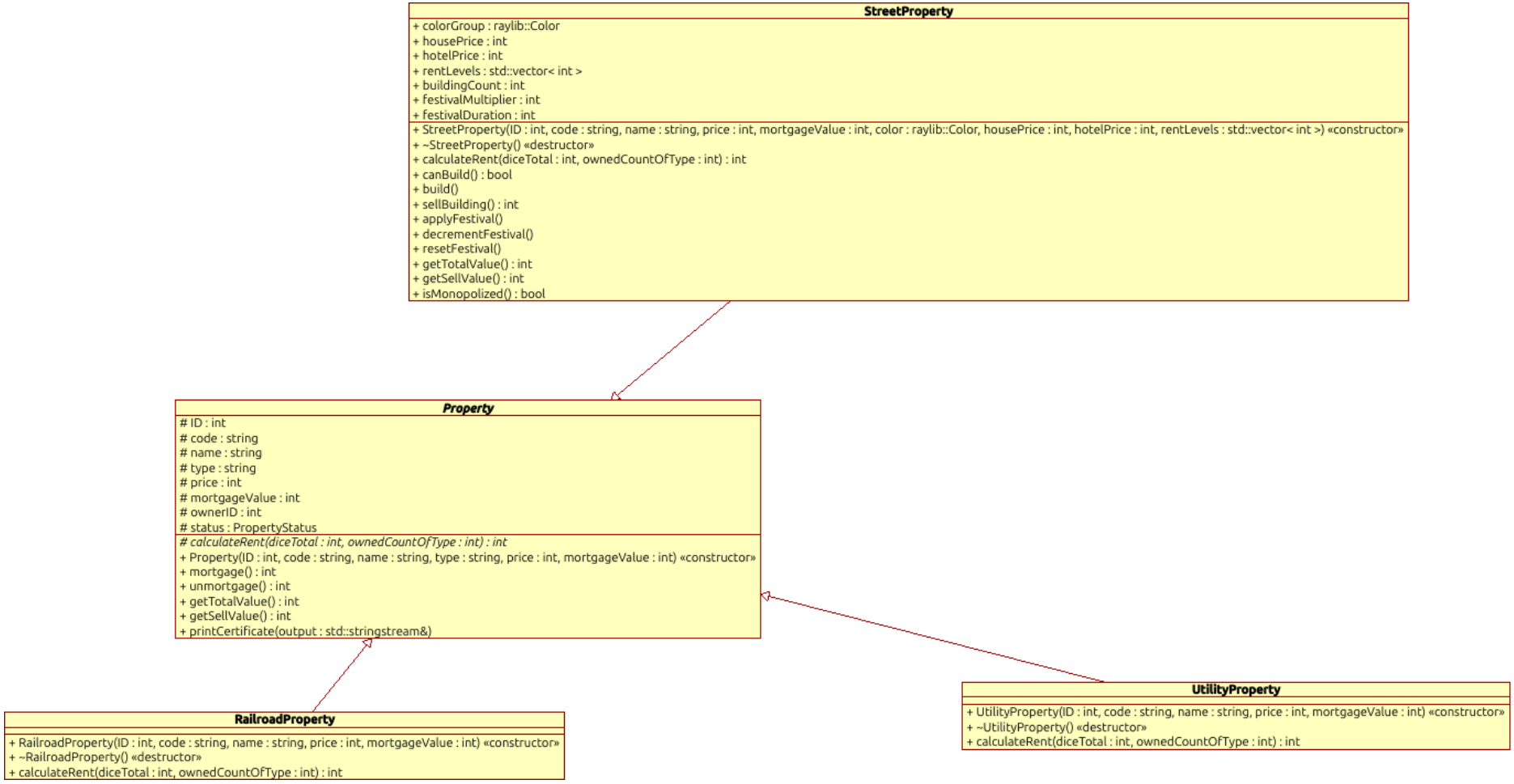


SSR - kata Fatih terserah sih

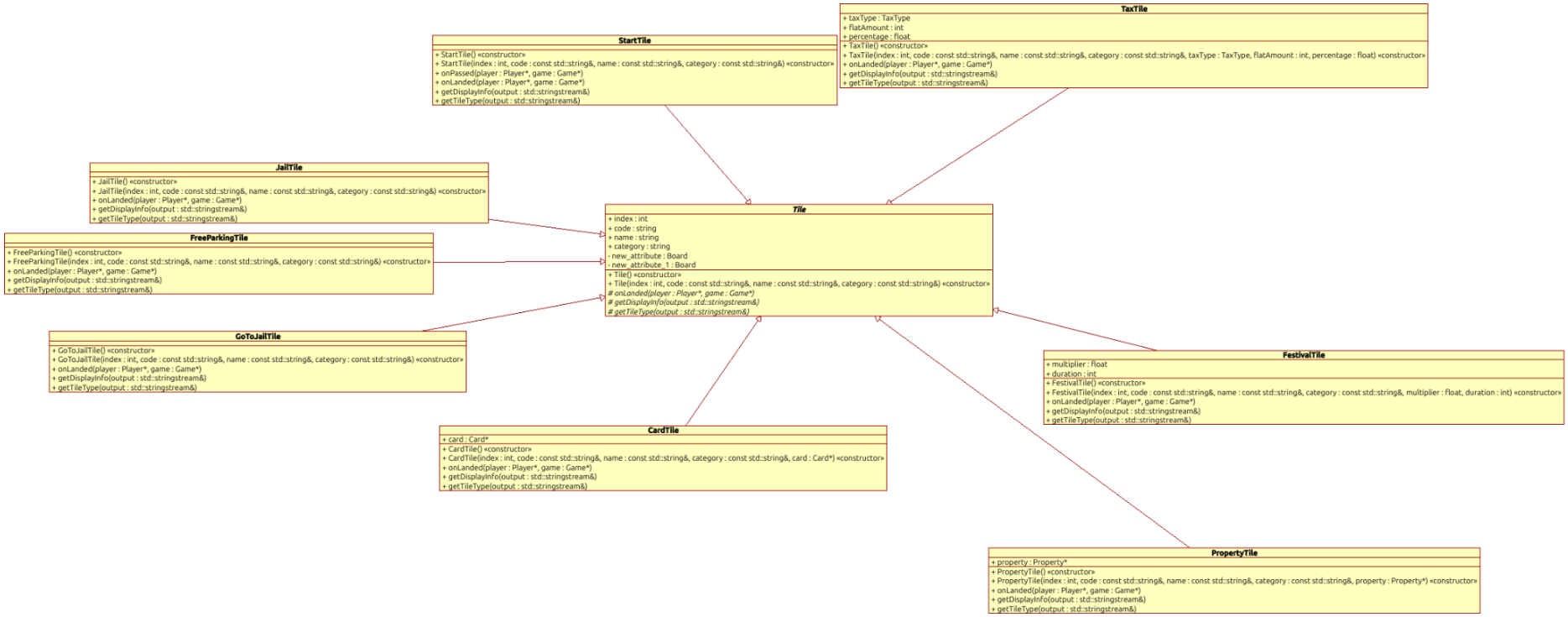
1. Diagram Kelas



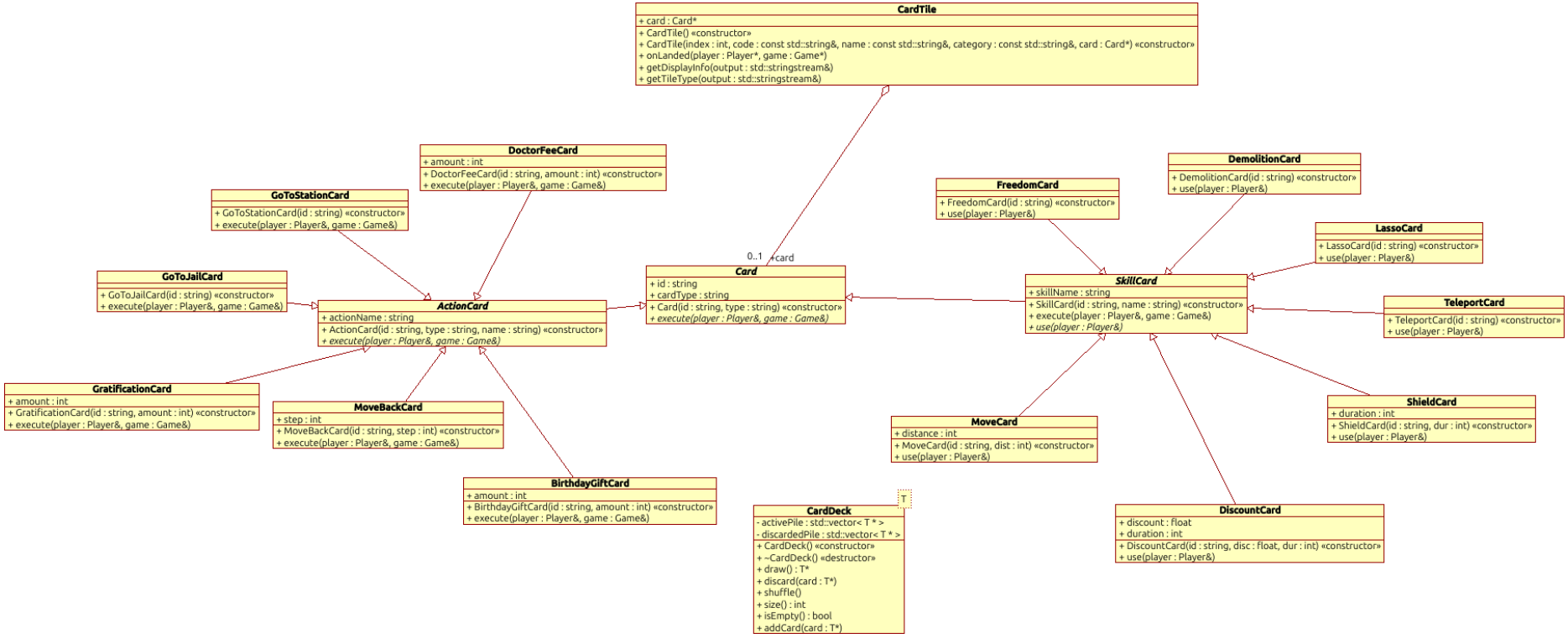
Gambar 1.1 Hubungan Diagram Kelas dilihat dari sekitar Kelas Game



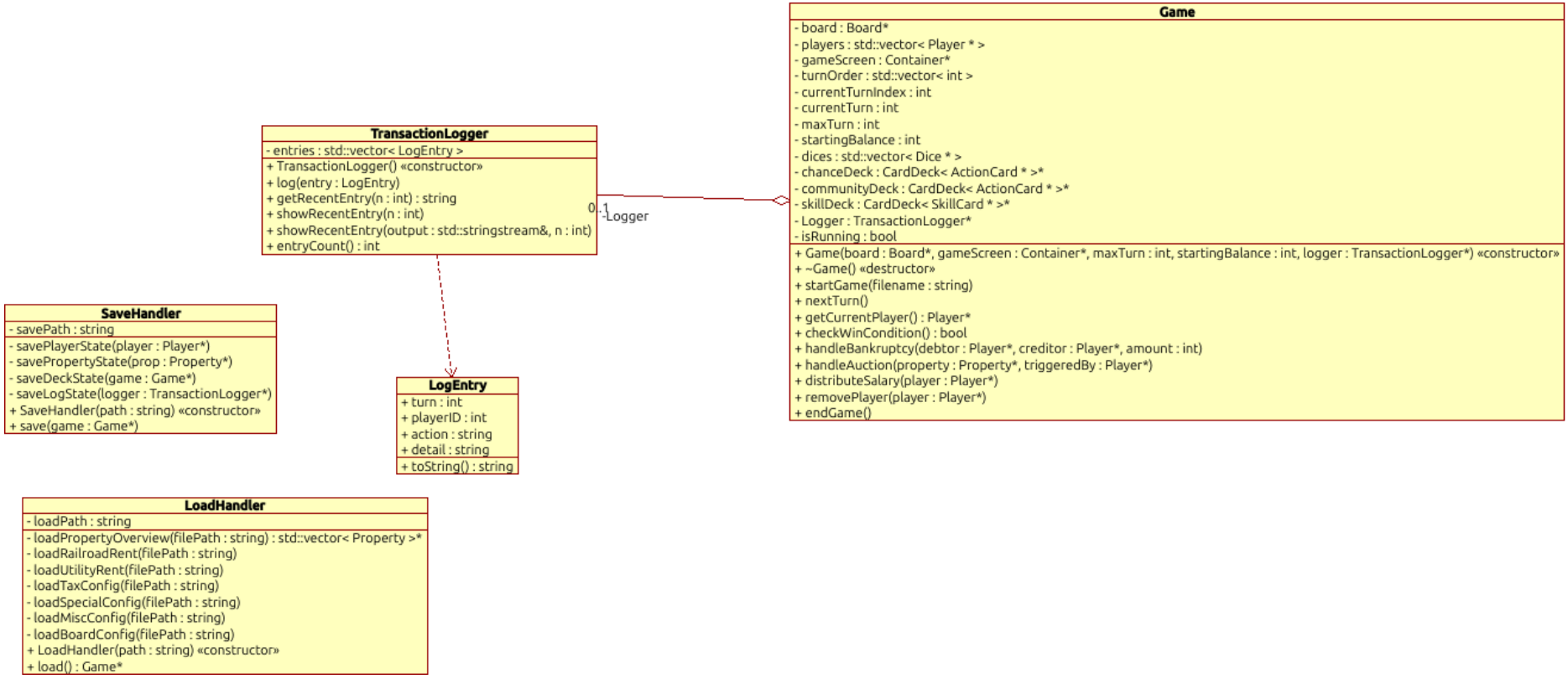
Gambar 1.2 Hubungan Diagram Kelas dilihat dari sekitar Kelas Property



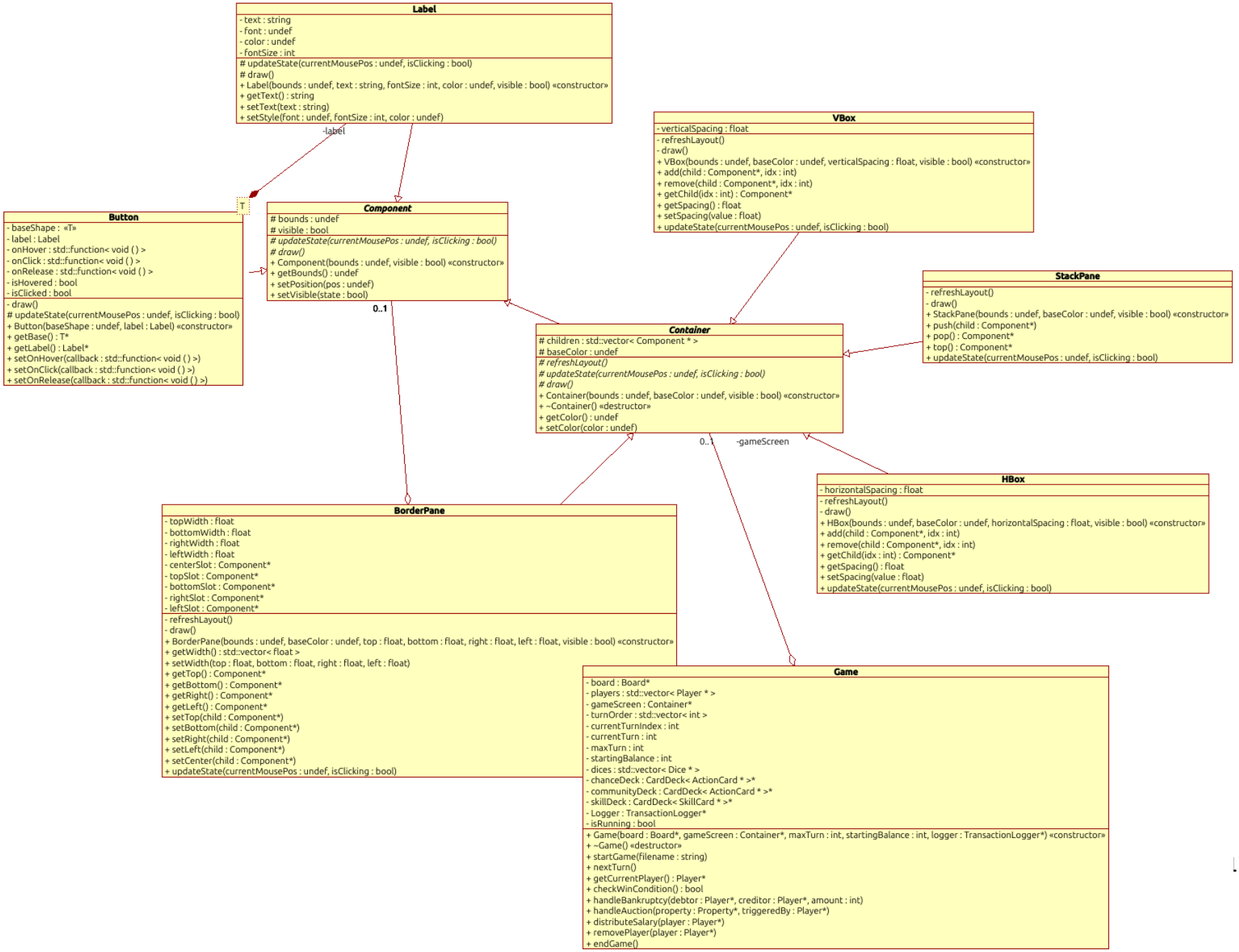
Gambar 1.3 Hubungan Diagram Kelas dilihat dari sekitar Kelas Tile



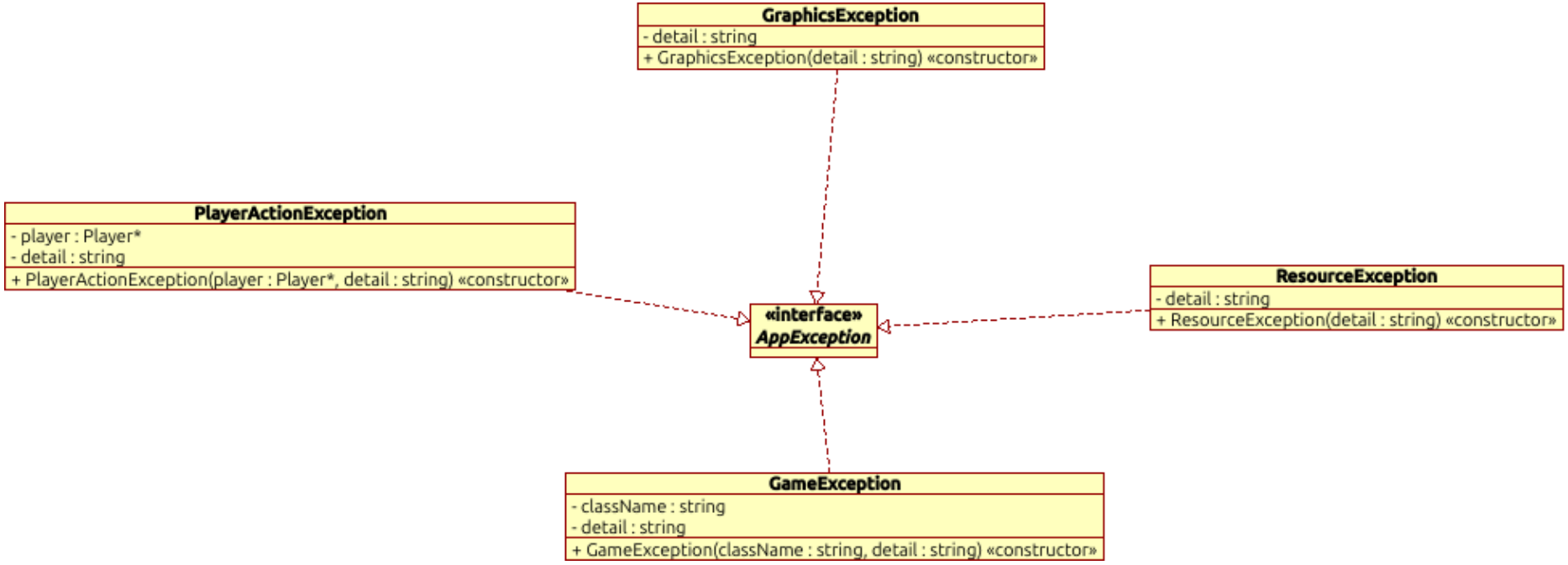
Gambar 1.4 Hubungan Diagram Kelas dilihat dari sekitar Kelas Card



Gambar 1.5 Hubungan Diagram Kelas dilihat dari sekitar Kelas TransactionLogger, LoadHandler, dan SaveHandler


Label
- text : string
- font : undef
- color : undef
- fontSize : int
updateState(currentMousePos : undef, isClicking : bool)
draw()
+ Label(bounds : undef, text : string, fontSize : int, color : undef, visible : bool) «constructor»
+ getText() : string
+ setText(text : string)
+ setStyle(font : undef, fontSize : int, color : undef)
-label
T
Button
- baseShape : «T»
- label : Label
- onHover : std::function< void ( ) >
- onClick : std::function< void ( ) >
- onRelease : std::function< void ( ) >
- isHovered : bool
- isClicked : bool
- draw()
updateState(currentMousePos : undef, isClicking : bool)
+ Button(baseShape : undef, label : Label) «constructor»
+ getBase() : T*
+ getLabel() : Label*
+ setOnHover(callback : std::function< void ( ) >)
+ setOnClick(callback : std::function< void ( ) >)
+ setOnRelease(callback : std::function< void ( ) >)
Component
bounds : undef
visible : bool
updateState(currentMousePos : undef, isClicking : bool)
draw()
+ Component(bounds : undef, visible : bool) «constructor»
+ getBounds() : undef
+ setPosition(pos : undef)
+ setVisible(state : bool)
0..1
VBox
- verticalSpacing : float
- refreshLayout()
- draw()
+ VBox(bounds : undef, baseColor : undef, verticalSpacing : float, visible : bool) «constructor»
+ add(child : Component*, idx : int)
+ remove(child : Component*, idx : int)
+ getChild(idx : int) : Component*
+ getSpacing() : float
+ setSpacing(value : float)
+ updateState(currentMousePos : undef, isClicking : bool)
StackPane
- refreshLayout()
- draw()
+ StackPane(bounds : undef, baseColor : undef, visible : bool) «constructor»
+ push(child : Component*)
+ pop() : Component*
+ top() : Component*
+ updateState(currentMousePos : undef, isClicking : bool)
Container
children : std::vector< Component * >
baseColor : undef
refreshLayout()
updateState(currentMousePos : undef, isClicking : bool)
draw()
+ Container(bounds : undef, baseColor : undef, visible : bool) «constructor»
+ ~Container() «destructor»
+ getColor() : undef
+ setColor(color : undef)
0..1
-gameScreen
HBox
- horizontalSpacing : float
- refreshLayout()
- draw()
+ HBox(bounds : undef, baseColor : undef, horizontalSpacing : float, visible : bool) «constructor»
+ add(child : Component*, idx : int)
+ remove(child : Component*, idx : int)
+ getChild(idx : int) : Component*
+ getSpacing() : float
+ setSpacing(value : float)
+ updateState(currentMousePos : undef, isClicking : bool)
BorderPane
- topWidth : float
- bottomWidth : float
- rightWidth : float
- leftWidth : float
- centerSlot : Component*
- topSlot : Component*
- bottomSlot : Component*
- rightSlot : Component*
- leftSlot : Component*
- refreshLayout()
- draw()
+ BorderPane(bounds : undef, baseColor : undef, top : float, bottom : float, right : float, left : float, visible : bool) «constructor»
+ getWidth() : std::vector< float >
+ setWidth(top : float, bottom : float, right : float, left : float)
+ getTop() : Component*
+ getBottom() : Component*
+ getRight() : Component*
+ getLeft() : Component*
+ setTop(child : Component*)
+ setBottom(child : Component*)
+ setRight(child : Component*)
+ setLeft(child : Component*)
+ setCenter(child : Component*)
+ updateState(currentMousePos : undef, isClicking : bool)
Game
- board : Board*
- players : std::vector< Player * >
- gameScreen : Container*
- turnOrder : std::vector< int >
- currentTurnIndex : int
- currentTurn : int
- maxTurn : int
- startingBalance : int
- dices : std::vector< Dice * >
- chanceDeck : CardDeck< ActionCard * >*
- communityDeck : CardDeck< ActionCard * >*
- skillDeck : CardDeck< SkillCard * >*
- Logger : TransactionLogger*
- isRunning : bool
+ Game(board : Board*, gameScreen : Container*, maxTurn : int, startingBalance : int, logger : TransactionLogger*) «constructor»
+ ~Game() «destructor»
+ startGame(filename : string)
+ nextTurn()
+ getCurrentPlayer() : Player*
+ checkWinCondition() : bool
+ handleBankruptcy(debtor : Player*, creditor : Player*, amount : int)
+ handleAuction(property : Property*, triggeredBy : Player*)
+ distributeSalary(player : Player*)
+ removePlayer(player : Player*)
+ endGame()

Gambar 1.6 Hubungan Diagram Kelas dilihat dari sekitar Kelas Component



Gambar 1.7 Hubungan Diagram Kelas dilihat dari sekitar Kelas AppException

2. Deskripsi Kelas

2.1. Kelas GameApp

elas GameApp merupakan kelas entry point yang mengatur alur program dari awal hingga akhir. Kelas ini bertanggung jawab untuk menampilkan menu utama, memuat konfigurasi (memanggil LoadHandler), memilih apakah akan memulai permainan baru atau memuat dari save, serta menjalankan sesi permainan (GameSession). Hanya ada satu objek GameApp sepanjang eksekusi program.

Terdapat atribut:

- [private] gameTitle : string – judul permainan
- [private] gameDescription : string – deskripsi permainan
- Terdapat metode:
- [public] start() – memulai aplikasi, menampilkan menu, dan menjalankan sesi permainan
- [private] printStartArt() – menampilkan ASCII art awal
- [private] printEndArt() – menampilkan ASCII art akhir

Catatan: GameApp memiliki atribut static GameSession* currentSession yang menyimpan pointer sesi aktif saat ini, digunakan oleh kelas lain (misal kartu) untuk mengakses GameSession tanpa passing parameter berantai.

2.2. Kelas GameSession

Kelas GameSession teragregasi dari kelas Player, Board, Dice, CardDeck, dan TransactionLogger. Kelas ini merupakan desain objek yang menyimpan informasi terkait keadaan suatu sesi permainan yang sedang aktif. Pada setiap saat hanya boleh ada satu objek dari kelas GameSession, dan setiap informasi dari objek GameSession akan diakses oleh kelas-kelas lainnya untuk mendapatkan informasi yang dibutuhkan dalam melakukan suatu operasi. Kelas ini merupakan pusat kendali utama permainan.

Terdapat atribut:

- [private] players : vector<Player*> – daftar pemain dalam permainan
- [private] currentPlayerIndex : int – indeks pemain yang sedang bergilir

- [private] currentTurn : int – nomor giliran saat ini
- [private] maxTurn : int – batas maksimum giliran (dari konfigurasi)
- [private] startingBalance : int – saldo awal pemain (dari konfigurasi)
- [private] dices : Dice – objek dadu yang digunakan
- [private] logger : TransactionLogger* – pencatat log transaksi
- [private] config : GameConfig* – konfigurasi permainan
- [private] isRunning : bool – status apakah permainan masih berjalan

Terdapat metode:

- [public] startGame() – memulai loop utama permainan
- [public] runCommand(command: string&) : int – mengeksekusi perintah dari pemain
- [public] nextTurn() – melanjutkan ke giliran berikutnya dengan dadu acak
- [public] nextTurn(die1: int, die2: int) – melanjutkan giliran dengan hasil dadu tertentu
- [public] getCurrentPlayer() : Player* – mengembalikan pemain yang sedang bergilir
- [public] hasWinner() : bool – mengecek apakah permainan sudah memiliki pemenang
- [public] checkWinCondition() : bool – mengecek kondisi kemenangan
- [public] handleBankruptcy(debtor: Player*, creditor: Player*, amount: int) – menangani kebangkrutan pemain
- [public] handleAuction(property: Property*, triggeredBy: Player*) – menjalankan lelang properti
- [public] handleGadai(player: Player*) – menangani perintah gadai
- [public] handleTebus(player: Player*) – menangani perintah tebus
- [public] handleBangun(player: Player*) – menangani perintah bangun
- [public] useSkillCard(player: Player*) – menangani penggunaan kartu kemampuan
- [public] drawCard(player: Player*) – mengambil kartu dari deck (Kesempatan/Dana Umum)
- [public] distributeSalary(player: Player*) – memberikan gaji saat melewati GO
- [public] removePlayer(player: Player*) – menghapus pemain yang bangkrut dari permainan
- [public] endGame() – mengakhiri permainan dan menampilkan pemenang

2.3. Kelas GameConfig

Kelas GameConfig merupakan desain objek yang menyimpan seluruh konfigurasi yang dibaca dari file konfigurasi (property, pajak, special, misc, railroad, utility, dan susunan papan). Kelas ini digunakan oleh GameSession dan LoadHandler. Kelas ini tidak memiliki metode kompleks, hanya berfungsi sebagai wadah data.

Terdapat atribut:

- [public] maxTurn : int – batas maksimum giliran
- [public] initialBalance : int – saldo awal pemain
- [public] pphFlat : int – nilai flat Pajak Penghasilan
- [public] pphPercentage : int – persentase Pajak Penghasilan
- [public] pbmFlat : int – nilai flat Pajak Barang Mewah
- [public] goSalary : int – gaji saat melewati GO
- [public] jailFine : int – denda keluar dari penjara
- [public] board : Board* – objek papan permainan yang sudah diinisialisasi
- [public] colorMap : unordered_map<string, raylib::Color> – pemetaan nama warna ke nilai warna Raylib
- Terdapat metode:
- [public] getBoard() : Board* – mengembalikan pointer ke papan permainan

2.4. Kelas Board

Kelas Board terkomposisi oleh kelas Tile dan akan mengagregasi kelas Game. Kelas ini merupakan desain objek representasi logikal (*core*) dari papan permainan.

Terdapat atribut:

- [private] tiles : vector<Tile*>
- [private] tileMap : map <string, Tile*>
- [private] totalTiles : int

Terdapat metode:

- [public] getTileCount() : int
- [public] getTile(index: int) : Tile*
- [public] getTileByCode(code: string) : Tile*
- [public] getNextTileIndex(current: int, steps: int) : int
- [public] findNearestStation(fromIndex: int) : int
- [public] getJailIndex() : int
- [public] getStartTileIndex() : int
- [public] setTile(tile: Tile*, idx: int)

Kontrak kelas ini berisi berbagai operasi dalam menentukan letak tile berdasarkan properti tertentu yang diinginkan.

2.5. Kelas Dice

Kelas Dice merupakan representasi objek dadu dalam permainan Nimonspoli. Dadu yang digunakan berjumlah 2 buah, masing-masing bersisi 6 (nilai 1–6). Kelas ini dapat menghasilkan lemparan acak menggunakan fungsi pembangkit bilangan acak, atau dapat diatur secara manual (untuk keperluan debug atau perintah ATUR_DADU).

Terdapat atribut:

- [private] num1 : int – nilai dadu pertama
- [private] num2 : int – nilai dadu kedua
- [private] randFunc : function<int(int,int)> – fungsi pembangkit bilangan acak (dapat dikustomisasi)
- Terdapat metode:
- [public] roll() – melempar kedua dadu secara acak
- [public] setManual(x: int, y: int) – mengatur nilai dadu secara manual
- [public] getTotal() : int – mengembalikan jumlah kedua dadu
- [public] isDouble() : bool – mengembalikan true jika kedua dadu bernilai sama
- [public] getDie1() : int – mengembalikan nilai dadu pertama
- [public] getDie2() : int – mengembalikan nilai dadu kedua

2.6. Kelas Player

Kelas Player mengagregasi kelas Game, dan merupakan kelas representasi objek pemain dalam permainan Nimonspoli, kelas ini akan menyimpan seluruh data dan keadaan pemain yang dibutuhkan untuk melakukan operasi/*command* permainan.

Terdapat atribut:

- [protected] username : string
- [protected] money : int
- [protected] position : int
- [protected] status : PlayerStatus
- [protected] properties : vector<Property *>
- [protected] hand : vector<SkillCard *>
- [protected] playerIndex : int
- [protected] isShieldActive : bool
- [protected] discount : int
- [protected] jailAttempts : int
- [protected] hasRolledDice : bool
- [protected] hasUsedSkill : bool
- [protected] consecutiveDoubles : int

Dengan tipe enumerasi tambahan:

- [public enum] PlayerStatus: {ACTIVE, BANKRUPT, JAILED}

Terdapat metode:

- [public] moveTo(position: int)
- [public] moveForward(steps: int, board: Board*): int
- [public] addProperty(property: Property*)
- [public] removeProperty(property: Property*)
- [public] addCard(card: SkillCard*)
- [public] removeCard(index: int): SkillCard*

- [public] getCardCount(): int
- [public] getTotalWealth(): int
- [public] getPropertiesByColor(color: raylib::Color): vector<Property *>
- [public] getOwnedRailroadCount(): int
- [public] getOwnedUtilityCount(): int
- [public] canAfford(amount: int): bool
- [public] getMaxLiquidationValue(): int
- [public] printProperties()
- [public] isActive(): bool
- [public] isBankrupt(): bool
- [public] isJailed(): bool
- [pure virtual protected] chooseInput(choices: vector<int>): int

2.7. Kelas HumanPlayer

Kelas HumanPlayer adalah turunan langsung dari kelas Player yang merepresentasikan pemain asli.

Terdapat method:

- [public override] chooseInput(choices: vector<int>): int

Kelas ini hanya menambahkan *override* untuk interaksi yang meminta pilihan kepada pemain, implementasi akan disesuaikan dengan desain antarmuka nantinya.

2.8. Kelas BotPlayer

Kelas BotPlayer adalah turunan langsung dari kelas Player yang merepresentasikan pemain komputer (COM).

Tidak terdapat atribut spesifik kelas.

Terdapat method:

- [public override] chooseInput(choices: vector<int>): int

Kelas ini hanya menambahkan *override* untuk interaksi yang meminta pilihan kepada pemain, implementasi akan disesuaikan dengan desain strategi bot nantinya.

2.9. Kelas Property

Kelas Property merupakan kelas dasar (basis) dengan sifat abstrak dari setiap objek properti permainan. Kelas ini mengagregasi kelas Player dan menyimpan semua informasi dasar dan kontrak yang dibutuhkan untuk melakukan operasi pada properti.

Terdapat atribut:

- [protected] ID : int
- [protected] code : string
- [protected] name : string
- [protected] type : string
- [protected] price : int
- [protected] mortgageValue : int
- [protected] ownerID : int
- [protected] status : PropertyStatus

Dengan tipe enumerasi tambahan:

- [public enum] PropertyStatus: {BANK, OWNED, MORTGAGED}

Terdapat metode:

- [pure virtual protected] calculateRent(diceTotal: int, ownedCountOfType: int): int
- [public] mortgage(): int
- [public] unmortgage(): int

- [public] getTotalValue(): int
- [public] getSellValue(): int
- [public] printCertificate(output: std::stringstream&)

2.10. Kelas StreetProperty

Kelas StreetProperty merupakan kelas turunan langsung dari kelas Property yang menyimpan informasi dari properti dengan jenis lahan.

Terdapat atribut:

- [public] colorGroup: raylib::Color
- [public] housePrice: int
- [public] hotelPrice: int
- [public] rentLevels: vector<int>
- [public] buildingCount: int
- [public] festivalMultiplier: int
- [public] festivalDuration: int

Terdapat metode:

- [public override] calculateRent(diceTotal: int, ownedCountOfType: int): int
- [public] canBuild(): bool
- [public] build()
- [public] sellBuilding(): int
- [public] applyFestival()
- [public] decrementFestival()
- [public] resetFestival()
- [public] getTotalValue(): int
- [public] getSellValue(): int

- [public] isMonopolized(): bool

Kontrak yang disediakan oleh StreetProperty berhubungan dengan operasi-operasi pembangungan properti, penjualan, dan hal-hal terkait festival.

2.11. Kelas RailroadProperty

Kelas RailroadProperty merupakan kelas turunan langsung dari kelas Property yang menyimpan informasi dari properti dengan jenis stasiun.

Tidak terdapat atribut spesifik kelas.

Terdapat metode:

- [public override] calculateRent(diceTotal: int, ownedCountOfType: int): int

2.12. Kelas UtilityProperty

Kelas UtililtyProperty merupakan kelas turunan langsung dari kelas Property yang menyimpan informasi dari properti dengan jenis PLN atau PAM.

Tidak terdapat atribut spesifik kelas.

Terdapat metode:

- [public override] calculateRent(diceTotal: int, ownedCountOfType: int): int

2.13. Kelas Tile

Kelas Tile merupakan kelas dasar (basis) dengan sifat abstrak dari setiap objek petak permainan. Kelas ini mengkomposisi kelas Board dan menyimpan semua informasi dasar dan kontrak yang dibutuhkan untuk melakukan operasi pada petak.

Terdapat atribut:

- [public] index: int
- [public] code: string
- [public] name: string
- [public] category: string

Terdapat metode:

- [pure virtual protected] onLanded(player: Player*, game: Game*)
- [pure virtual protected] getDisplayInfo(output: std::stringstream&)
- [pure virtual protected] getTileType(output: std::stringstream)

Kelas ini mendefinisikan antarmuka yang harus dipenuhi oleh kelas turunan, antarmuka onLanded() mendefinisikan aksi yang dilakukan terhadap player ketika berhenti di suatu petak, getDisplayInfo() akan memberikan informasi lengkap terkait tile ke suatu *stream* output, getTileType akan memberikan tipe petak dalam bentuk string ke suatu stream output. Metode-metode diterapkan dengan prinsip polimorfisme.

2.14. Kelas CardTile

Kelas CardTile merupakan turunan langsung dari kelas Tile yang menggambarkan objek petak kartu.

Terdiri dari atribut:

- [public] card: Card*

Terdapat metode:

- [public override] onLanded(player: Player*, game: Game*)
- [public override] getDisplayInfo(output: std::stringstream&)
- [public override] getTileType(output: std::stringstream&)

Kelas ini menyimpan atribut berupa tipe kartu dengan prinsip polimorfisme dan memenuhi kontrak antarmuka dengan kelas dasar Tile.

2.15. Kelas FestivalTile

Kelas FestivalTile merupakan turunan langsung dari kelas Tile yang menggambarkan objek petak festival.

Terdiri dari atribut:

- [public] multiplier: float
- [public] duration: int

Terdapat metode:

- [public override] onLanded(player: Player*, game: Game*)
- [public override] getDisplayInfo(output: std::stringstream&)
- [public override] getTileType(output: std::stringstream&)

Kelas ini menyimpan atribut terkait informasi festival dan memenuhi kontrak antarmuka dengan kelas dasar Tile.

2.16. Kelas TaxTile

Kelas TaxTile merupakan turunan langsung dari kelas Tile yang menggambarkan objek petak pajak.

Terdiri dari atribut:

- [public] taxType: TaxType

- [public] flatAmount: int
- [public] percentage: float

Dengan tipe enumerasi tambahan:

- [public enum] TaxType: {TAX_PPH, TAX_PBM}

Terdapat method:

- [public override] onLanded(player: Player*, game: Game*)
- [public override] getDisplayInfo(output: std::stringstream&)
- [public override] getTileType(output: std::stringstream&)

Kelas ini menyimpan atribut terkait informasi pajak dan memenuhi kontrak antarmuka dengan kelas dasar Tile.

2.17. Kelas StartTile

Kelas StartTile merupakan turunan langsung dari kelas Tile yang menggambarkan objek petak mulai (GO).

Tidak memiliki atribut spesifik kelas.

Terdapat method:

- [public] onPassed(player: Player*, game: Game*)
- [public override] onLanded(player: Player*, game: Game*)
- [public override] getDisplayInfo(output: std::stringstream&)
- [public override] getTileType(output: std::stringstream&)

Kelas ini memenuhi kontrak antarmuka dengan kelas dasar Tile serta memiliki metode spesifik onPassed() yang memberikan perilaku jika pemain melewati petak tersebut tetapi tidak berhenti disana.

2.18. Kelas JailTile

Kelas JailTile merupakan turunan langsung dari kelas Tile yang menggambarkan objek petak penjara.

Tidak memiliki atribut spesifik kelas.

Terdapat method:

- [public override] onLanded(player: Player*, game: Game*)
- [public override] getDisplayInfo(output: std::stringstream&)
- [public override] getTileType(output: std::stringstream&)

Kelas ini memenuhi kontrak antarmuka dengan kelas dasar Tile.

2.19. Kelas GoToJailTile

Kelas GoToJailTile merupakan turunan langsung dari kelas Tile yang menggambarkan objek petak “pergi ke penjara”.

Tidak memiliki atribut spesifik kelas.

Terdapat method:

- [public override] onLanded(player: Player*, game: Game*)
- [public override] getDisplayInfo(output: std::stringstream&)
- [public override] getTileType(output: std::stringstream&)

Kelas ini memenuhi kontrak antarmuka dengan kelas dasar Tile.

2.20. Kelas FreeParkingTile

Kelas FreeParkingTile merupakan turunan langsung dari kelas Tile yang menggambarkan objek petak parkir bebas.

Tidak memiliki atribut spesifik kelas.

Terdapat method:

- [public override] onLanded(player: Player*, game: Game*)
- [public override] getDisplayInfo(output: std::stringstream&)
- [public override] getTileType(output: std::stringstream&)

Kelas ini memenuhi kontrak antarmuka dengan kelas dasar Tile.

2.21. Kelas PropertyTile

Kelas PropertyTile merupakan turunan langsung dari kelas yang menggambarkan objek petak properti.

Terdiri dari atribut:

- [public] Property: Property*

Terdapat method:

- [public override] onLanded(player: Player*, game: Game*)
- [public override] getDisplayInfo(output: std::stringstream&)
- [public override] getTileType(output: std::stringstream&)

Kelas ini menyimpan atribut terkait informasi properti dengan prinsip polimorfisme dan memenuhi kontrak antarmuka dengan kelas dasar Tile.

2.22. Kelas Card

Kelas Card merupakan kelas dasar yang merepresentasikan kartu dalam permainan yang memiliki efek tertentu terhadap pemain atau jalannya permainan.

Terdiri dari atribut:

- [public] ID : string
- [public] cardType : string

Terdapat method:

- [pure virtual protected] execute(player: Player&, game: Game&)

Kelas ini berperan sebagai abstraksi umum bagi seluruh jenis kartu dalam sistem permainan. Dengan menyediakan method execute, kelas ini mendefinisikan kontrak bahwa setiap kartu harus memiliki perilaku yang dapat memengaruhi state permainan. Implementasi efek kartu dilakukan melalui interaksi dengan objek Player dan Game, sehingga seluruh perubahan state tetap berada pada layer logika permainan dan tidak bergantung pada GUI. Desain ini memanfaatkan prinsip polymorphism sehingga setiap turunan kartu dapat memiliki perilaku spesifik masing-masing.

2.23. Kelas ActionCard

Kelas ActionCard merupakan turunan dari Card yang merepresentasikan kartu dengan efek langsung.

Terdiri dari atribut:

- [public] actionName : string

Terdapat method:

- [pure virtual protected] execute(player: Player&, game: Game&)

Kelas ini digunakan untuk merepresentasikan kartu yang efeknya langsung dijalankan saat kartu digunakan atau diperoleh oleh pemain. ActionCard berperan dalam memodifikasi state permainan secara instan, seperti perubahan posisi pemain, penambahan atau

pengurangan uang, maupun perubahan status tertentu. Dengan mengoverride method execute, setiap turunan ActionCard dapat mengimplementasikan perilaku spesifiknya masing-masing sehingga mendukung extensibility sistem kartu.

2.24. Kelas GoToStationCard

Kelas GoToStationCard merupakan turunan dari ActionCard yang berfungsi untuk memindahkan pemain ke stasiun terdekat pada papan permainan.

Terdapat method:

- [public override] execute(player: Player&, game: Game&)

Dalam pelaksanaannya, kelas ini memanfaatkan relasi dengan objek Game untuk mengakses Board dan menentukan lokasi stasiun terdekat dari posisi pemain saat ini. Desain ini menunjukkan pemanfaatan kolaborasi antar kelas untuk menghasilkan efek kartu yang kontekstual terhadap state permainan.

2.25. Kelas MoveBackCard

Kelas MoveBackCard merupakan turunan dari ActionCard yang digunakan untuk memindahkan pemain mundur sejumlah langkah tertentu.

Terdiri dari atribut:

- [public] step : int

Terdapat method:

- [public override] execute(player: Player&, game: Game&)

Method execute akan memanfaatkan fungsi pergerakan pada objek Player dengan mempertimbangkan aturan papan permainan. Kelas ini berkontribusi dalam variasi mekanisme pergerakan yang dapat mempengaruhi strategi pemain.

2.26. Kelas GoToJailCard

Kelas GoToJailCard merupakan turunan dari ActionCard yang berfungsi untuk mengirim pemain ke penjara.

Tidak memiliki atribut kelas spesifik.

Terdapat method:

- [public override] execute(player: Player&, game: Game&)

Kelas ini akan mengubah posisi pemain ke posisi penjara serta mengubah status pemain menjadi jailed. Implementasi efek ini dilakukan dengan memanfaatkan informasi posisi penjara dari Board melalui Game. Kelas ini menunjukkan bagaimana kartu dapat memengaruhi tidak hanya posisi tetapi juga status pemain dalam permainan.

2.27. Kelas GratificationCard

Kelas GratificationCard merupakan turunan dari ActionCard yang memberikan keuntungan finansial kepada pemain.

Terdiri dari atribut:

- [public] amount : int

Terdapat method:

- [public override] execute(player: Player&, game: Game&)

Method execute akan menambahkan nilai amount ke saldo pemain. Kelas ini berfungsi sebagai mekanisme reward yang dapat meningkatkan sumber daya pemain dalam permainan.

2.28. Kelas BirthdayGiftCard

Kelas BirthdayGiftCard merupakan turunan dari ActionCard yang melibatkan interaksi antar pemain dalam bentuk distribusi uang.

Terdiri dari atribut:

- [public] amount : int

Terdapat method:

- [public override] execute(player: Player&, game: Game&)

Kelas ini akan mengakses seluruh pemain melalui objek Game dan memaksa pemain lain untuk memberikan sejumlah uang kepada pemain yang menggunakan kartu.

2.29. Kelas DoctorFeeCard

Kelas DoctorFeeCard merupakan turunan dari ActionCard yang memberikan efek penalti berupa pengurangan uang pemain.

Terdiri dari atribut:

- [public] amount : int

Terdapat method:

- [public override] execute(player: Player&, game: Game&)

Jika pemain tidak mampu membayar, maka kondisi kebangkrutan akan ditangani oleh sistem melalui kelas Game sehingga memastikan konsistensi aturan permainan.

2.30. Kelas SkillCard

Kelas SkillCard merupakan turunan dari Card yang merepresentasikan kartu dengan efek strategis yang dapat disimpan dan digunakan pada waktu tertentu oleh pemain.

Terdiri dari atribut:

- [public] skillName : string

Terdapat method:

- [pure virtual protected] use(player: Player&)

Berbeda dengan ActionCard, efek SkillCard tidak langsung dijalankan saat diperoleh, melainkan disimpan oleh pemain dan dapat digunakan sesuai kondisi permainan. Hal ini memberikan fleksibilitas strategi bagi pemain.Contoh implementasi SkillCard antara lain MoveCard, DiscountCard, ShieldCard, TeleportCard, LassoCard, DemolitionCard, dan FreedomCard.

2.31. Kelas MoveCard

Kelas MoveCard merupakan turunan dari SkillCard yang memungkinkan pemain untuk berpindah posisi secara strategis.

Terdiri dari atribut:

- [public] distance : int

Terdapat method:

- [public override] use(player: Player&)

Kelas ini memungkinkan pemain mengatur posisi secara fleksibel tanpa bergantung pada hasil dadu.

2.32. Kelas DiscountCard

Kelas DiscountCard merupakan turunan dari SkillCard yang memberikan keuntungan berupa potongan harga dalam transaksi.

Terdiri dari atribut:

- [public] discount : float
- [public] duration : int

Terdapat method:

- [public override] use(player: Player&)

Efek diskon akan digunakan oleh sistem lain seperti Property dalam perhitungan biaya sehingga menunjukkan keterkaitan antarkomponen dalam sistem.

2.33. Kelas ShieldCard

Kelas ShieldCard merupakan turunan dari SkillCard yang memberikan perlindungan kepada pemain dari efek negatif tertentu.

Terdiri dari atribut:

- [public] duration : int

Terdapat method:

- [public override] use(player: Player&)

Status perlindungan akan disimpan pada objek Player dan digunakan dalam berbagai interaksi yang berpotensi merugikan pemain.

2.34. Kelas TeleportCard

Kelas TeleportCard merupakan turunan dari SkillCard yang memungkinkan pemain berpindah ke lokasi tertentu secara instan.

Tidak memiliki atribut kelas spesifik.

Terdapat method:

- [public override] use(player: Player&)

Efek ini memberikan fleksibilitas tinggi dalam menentukan posisi strategis pemain di papan permainan.

2.35. Kelas LassoCard

Kelas LassoCard merupakan turunan dari SkillCard yang memungkinkan pemain untuk mempengaruhi atau mengambil aset pemain lain.

Tidak memiliki atribut kelas spesifik.

Terdapat method:

- [public override] use(player: Player&)

Kelas ini melibatkan interaksi antarobjek Player sehingga meningkatkan aspek kompetitif dalam permainan.

2.36. Kelas DemolitionCard

Kelas DemolitionCard merupakan turunan dari SkillCard yang digunakan untuk menghancurkan bangunan pada properti milik pemain lain.

Tidak memiliki atribut kelas spesifik.

Terdapat method:

- [public override] use(player: Player&)

Kelas ini akan menghapus seluruh bangunan pada properti target tanpa mengubah status kepemilikan (tetap OWNED). Jika properti tidak memiliki bangunan, maka kartu tidak memberikan efek. Efek ini dapat menyebabkan ketidakseimbangan jumlah bangunan dalam satu color group sehingga pemain harus melakukan penyesuaian sebelum dapat membangun kembali sesuai aturan pemerataan.'

2.37. Kelas FreedomCard

Kelas FreedomCard merupakan turunan dari SkillCard yang merepresentasikan kartu "Bebas dari Penjara" yang dapat digunakan oleh pemain untuk keluar dari kondisi penjara tanpa harus menunggu giliran atau membayar denda. Kelas ini memberikan alternatif solusi bagi pemain yang berada dalam status jailed.

Tidak memiliki atribut kelas spesifik.

Terdapat method:

- [public override] use(player: Player&)

Saat method use dipanggil, kartu ini akan mengubah status pemain dari JAILED menjadi ACTIVE serta mereset atribut terkait penjara seperti jumlah percobaan keluar (jailAttempts) pada objek Player. Kartu ini hanya dapat digunakan ketika pemain berada dalam kondisi penjara sehingga implementasinya perlu melakukan validasi terhadap status pemain sebelum efek dijalankan. Setelah digunakan, kartu akan dihapus dari inventory pemain.

2.38. Kelas CardDeck<T>

Kelas CardDeck<T> merupakan kelas generik yang digunakan untuk mengelola kumpulan kartu dalam permainan dan berinteraksi langsung dengan kelas Game.

Terdiri dari atribut:

- [private] activePile : vector<T>
- [private] discardedPile : vector<T>

Terdapat method:

- [public] draw() : T*
- [public] discard()
- [public] shuffle()
- [public] size() : int
- [public] isEmpty() : bool

Kelas ini bertanggung jawab dalam mengatur siklus kartu, mulai dari pengambilan, pembuangan, hingga pengacakan ulang. Dengan penggunaan template, kelas ini dapat digunakan untuk berbagai jenis kartu secara fleksibel. Kelas ini digunakan oleh Game untuk mendistribusikan kartu kepada pemain selama permainan berlangsung.

2.39. Kelas LogEntry

Merupakan desain objek yang akan menyimpan informasi untuk sebuah entri log permainan, kelas LogEntry akan mengkomposisi kelas TransactionLogger.

Terdiri dari atribut:

- [public] turn : int

- [public] playerID : int
- [public] action : string
- [public] detail : string

Terdapat method:

- [public] toString(): string

Sebuah entri log akan memiliki informasi nomor giliran, ID pemain, aksi saat ini, dan detail tambahan. Semua informasi dapat diambil dalam bentuk serialisasi string.

2.40. Kelas TransactionLogger

Merupakan desain objek yang akan menyimpan informasi seluruh log permainan, kelas ini terkomposisi dari kelas LogEntry.

Terdiri dari atribut:

- [private] entries: vector<LogEntry>

Terdapat method:

- [public] log(entry: LogEntry)
- [public] getRecentEntry(n: int): string
- [public] showRecentEntry(n: int)
- [public] showRecentEntry(output: std::stringstream&, n: int)
- [public] entryCount(): int

Kelas dapat memiliki kontrak untuk memasukkan entri log permainan, mendapatkan bentuk serialisasi entri log dalam bentuk string, dan mendapatkan jumlah aksi yang dicatat pada log sejauh ini.

2.41. Kelas LoadHandler

Merupakan desain objek yang akan melakukan operasi *load* pada permainan melalui *data access layer*.

Terdiri dari atribut:

- [private] loadPath: string

Terdapat method:

- [public] load(): Game*
- [private] loadPropertyOverview(filePath: string): vector<Property> *
- [private] loadRailroadRent(filePath: string)
- [private] loadUtilityRent(filePath: string)
- [private] loadTaxConfig(filePath: string)
- [private] loadSpecialConfig(filePath: string)
- [private] loadMiscConfig(filePath: string)
- [private] loadBoardConfig(filePath: string)

Kelas dapat digunakan melalui metode load() yang akan memanggil berbagai submetode privat sesuai masing-masing jenis file yang akan di-*load*.

2.42. Kelas SaveHandler

Merupakan desain objek yang akan melakukan operasi *save* pada permainan melalui *data access layer*.

Terdiri dari atribut:

- [private] savePath: string

Terdapat method:

- [public] save(game: Game*)
- [private] savePlayerState(player: Player*)
- [private] savePropertyState(prop: Property*)
- [private] saveDeckState(game: Game*)
- [private] saveLogState(logger: TransactionLogger*)

Kelas dapat digunakan melalui metode save() yang akan memanggil berbagai submetode privat sesuai masing-masing jenis format objek yang akan di-*save*.

2.43. Kelas AppException

Kelas basis dari setiap *custom exception* yang akan digunakan pada *error handling* aplikasi permainan Nimonspoli. Sebagai basis Exception, kelas ini akan menjadi turunan langsung dari std::exception.

Tidak memiliki atribut kelas spesifik.

Terdapat method:

- [virtual protected override] what(): string

2.44. Kelas GameException

Kelas GameException merupakan turunan langsung dari AppException. Kelas ini menunjukkan *exception* yang terjadi akibat ada kesalahan yang terjadi pada alur permainan yang disebabkan oleh aplikasi permainan itu sendiri (tanpa campur tangan pemain), misalnya terdapat *null pointer*, *index out-of-bound*, dan lain-lain.

Terdapat atribut:

- [private] className: string

- [private] detail: string

Terdapat method:

- [public override] what(): string

Akan terdapat informasi terkait kelas dan detail yang berkaitan dengan error tersebut.

2.45. Kelas PlayerActionException

Kelas PlayerActionException merupakan turunan langsung dari AppException. Kelas ini menunjukkan *exception* yang terjadi akibat ada kesalahan yang terjadi pada alur permainan yang disebabkan oleh aksi pemain, misalnya uang tidak cukup untuk membeli/lelang, slot kartu penuh, dan lain-lain.

Terdapat atribut:

- [private] player: Player*
- [private] detail: string

Terdapat method:

- [public override] what(): string

Akan terdapat informasi terkait pemain dan detail yang berkaitan dengan error tersebut.

2.46. Kelas ResourceException

Kelas ResourceException merupakan turunan langsung dari AppException. Kelas ini menunjukkan *exception* yang terjadi akibat adanya kesalahan saat berinteraksi dengan sumber daya eksternal untuk permainan, misalnya *file config* tidak valid, tidak ditemukan file untuk *font*, tidak ditemukan gambar untuk GUI, dan lain-lain.

Terdapat atribut:

- [private] detail: string

Terdapat method:

- [public override] what(): string

Akan terdapat informasi detail terkait sumber daya yang berkaitan dengan error tersebut.

2.47. Kelas Auction

Kelas Auction merupakan kelas yang menangani proses lelang properti. Lelang terjadi ketika seorang pemain memutuskan untuk tidak membeli properti (atau tidak mampu membeli), atau ketika properti milik pemain yang bangkrut ke Bank dilelang. Kelas ini mengatur giliran bidding, validasi bid, dan menentukan pemenang.

Terdapat atribut:

- [private] property : Property* – properti yang dilelang
- [private] activePlayers : vector<Player*> – daftar pemain yang berpartisipasi (tidak bangkrut)
- [private] triggerIndex : int – indeks pemain yang memicu lelang (untuk menentukan urutan)
- [private] logger : TransactionLogger* – pencatat log
- [private] gameTurn : int – giliran permainan saat lelang berlangsung
- [private] currentHighBid : int – tawaran tertinggi saat ini
- [private] currentHighBidder : Player* – pemain dengan tawaran tertinggi
- [private] consecutivePasses : int – jumlah pass berturut-turut
- [private] atLeastOneBid : bool – apakah sudah ada setidaknya satu tawaran
- Terdapat metode:
- [public] run() : AuctionResult – menjalankan proses lelang hingga selesai
- [private] buildBidOrder() : vector<Player*> – menentukan urutan giliran bidding
- [private] requestAction(player: Player*) : int – meminta aksi (PASS / BID) dari pemain
- [private] requestForcedBid(player: Player*) : int – memaksa pemain yang wajib melakukan bid (sesuai spesifikasi)

- [private] printAuctionState() – menampilkan status lelang saat ini
- [private] logBid(bidder: Player*, amount: int) – mencatat aksi bid ke logger
- [private] logPass(passer: Player*) – mencatat aksi pass ke logger
- [private] logResult(result: AuctionResult) – mencatat hasil lelang ke logger

2.48. Kelas AuctionResult

Kelas AuctionResult adalah struktur data sederhana (bisa berupa struct atau kelas kecil) yang digunakan untuk mengembalikan hasil dari proses lelang. Tidak memiliki metode kompleks.

Terdapat atribut:

- [public] winner : Player* – pemenang lelang (nullptr jika tidak ada)
- [public] winningBid : int – harga tawaran pemenang
- [public] sold : bool – true jika properti terjual, false jika tidak

3. Justifikasi Desain

Desain sistem pada permainan Nimonspoli ini dibangun dengan mengacu pada prinsip-prinsip utama dalam pemrograman berorientasi objek (OOP), yaitu encapsulation, inheritance, polymorphism, dan abstraction. Secara umum, pendekatan desain yang digunakan berfokus pada pemisahan tanggung jawab yang jelas antar kelas (separation of concerns) serta memastikan setiap kelas memiliki peran spesifik yang sesuai dengan konsep single responsibility principle.

3.1. Alasan Pemilihan Desain

Pemilihan pendekatan OOP didasarkan pada kebutuhan untuk memodelkan dunia permainan ke dalam bentuk objek yang memiliki atribut dan perilaku masing-masing. Setiap elemen dalam permainan, seperti Player, Property, Tile, dan Card, direpresentasikan sebagai kelas tersendiri agar memiliki tanggung jawab yang terpisah dan jelas.

Desain ini juga menerapkan prinsip *separation of concerns*, di mana sistem dibagi menjadi beberapa layer utama:

1. Core Game Logic (Game, Player, Board, Tile, Property, Card)
2. Card System (ActionCard, SkillCard dan turunannya)
3. Persistence Layer (LoadHandler, SaveHandler)
4. Logging System (TransactionLogger, LogEntry)
5. GUI Layer (Component dan turunannya)
6. Exception Handling

Pembagian ini bertujuan untuk:

1. Setiap bagian sistem memiliki tanggung jawab yang terisolasi dan spesifik
2. Mempermudah pengembangan secara paralel dalam tim
3. Memudahkan maintenance dan debugging
4. Menghindari terjadinya *God Class* sesuai dengan batasan pada spesifikasi

Sebagai contoh, Kelas Game dirancang sebagai *orchestrator* yang mengatur jalannya permainan tanpa mengambil alih tanggung jawab kelas lain. Hal ini penting untuk menghindari *God Class*. Seluruh logika spesifik, seperti perhitungan sewa atau efek kartu, tetap didelegasikan ke kelas yang bersangkutan. Selain itu, desain ini juga memperhatikan constraint bahwa GUI tidak boleh mengubah game state secara langsung sehingga seluruh perubahan state tetap dilakukan pada layer logic (C++), sementara GUI hanya berperan sebagai media input/output.

3.2. Pemanfaatan Konsep OOP

Desain ini secara eksplisit memanfaatkan prinsip-prinsip utama OOP:

A. Abstraction

Digunakan pada kelas-kelas dasar seperti:

1. Tile
2. Card
3. Property
4. Component
5. AppException

Kelas-kelas ini mendefinisikan kontrak umum tanpa mengikat implementasi spesifik sehingga mempermudah pengembangan fitur baru tanpa mengubah struktur utama.

B. Inheritance

Digunakan secara luas untuk membangun hierarki kelas, seperti:

1. Player → HumanPlayer, BotPlayer
2. Property → StreetProperty, RailroadProperty, UtilityProperty
3. Tile → berbagai jenis tile
4. Card → ActionCard, SkillCard → turunan spesifik
5. Component → Button, Container, Label, dll

Pendekatan ini memungkinkan reuse kode dan memperjelas relasi antarentitas dalam sistem. Dengan inheritance, sistem menjadi lebih mudah dikembangkan dan menghindari duplikasi kode

C. Polymorphism

Digunakan pada:

1. execute() pada Card
2. use() pada SkillCard
3. onLanded() pada Tile
4. draw() dan updateState() pada GUI

Dengan polymorphism, sistem dapat menangani berbagai jenis objek dengan tipe berbeda secara uniform, tetapi tetap memiliki perilaku spesifik masing-masing sehingga meningkatkan fleksibilitas dan extensibility. Selain itu terdapat juga *static polymorphism* yang ditentukan saat *compile time* pada kelas CardDeck<T> dan Button<T> yang menggunakan *template* generik

D. Encapsulation

Setiap kelas mengelola datanya sendiri dan hanya mengekspos operasi yang diperlukan. Contohnya:

1. Player mengelola uang, properti, dan statusnya sendiri
2. Property mengelola perhitungan nilai dan sewa
3. CardDeck mengelola siklus kartu

Hal ini menjaga konsistensi state dan mencegah manipulasi data secara langsung dari luar. Tetapi tidak semua atribut objek otomatis didesain menjadi *private* ataupun *protected*. Terdapat beberapa kelas yang dianggap dapat diakses bebas dari luar karena data yang disimpan dari atribut tidak dianggap berbahaya dan telah ada lapisan enkapsulasi lain di dalamnya, misalnya pointer Property dari kelas PropertyTile bersifat *public* tetapi atribut dari Property tersebut tidak langsung bersifat *public* dan ID dari kelas Card bersifat *public* tetapi karena kelas tersebut adalah kelas abstrak maka aman karena hanya dapat diakses ketika kelas turunannya diinstansiasi

3.3. Kelebihan Desain

Desain yang diusulkan memiliki beberapa keunggulan utama:

1. Modularitas Tinggi
   Setiap komponen sistem dipisahkan secara jelas sehingga memudahkan pengembangan, pengujian, dan debugging.

2. Extensibility
   Sistem mudah dikembangkan dengan menambahkan:

   a. Jenis kartu baru (cukup extend Card atau SkillCard)

   b. Jenis tile baru

   c. Mekanisme permainan baru tanpa harus memodifikasi banyak bagian yang sudah ada.

3. Maintainability
   Perubahan pada satu bagian (misalnya GUI) tidak memengaruhi core logic karena adanya pemisahan layer.

4. Reusability
   Banyak komponen seperti CardDeck<T> dan Component bersifat generik dan dapat digunakan kembali dalam konteks lain.

5. Kesesuaian dengan Spesifikasi

   a. Tidak terdapat *God Class*

   b. Setiap objek memiliki behavior sendiri

   c. GUI tidak mengandung logic permainan

6. Scalability Konseptual
   Desain memungkinkan pengembangan lebih lanjut seperti penambahan mode multiplayer, AI yang lebih kompleks atau fitur tambahan tanpa perubahan besar pada struktur inti.

3.4. Kekurangan Desain

Meskipun memiliki banyak keunggulan, desain ini juga memiliki beberapa kekurangan:

1. Kompleksitas Tinggi
   Jumlah kelas yang banyak dan relasi yang kompleks dapat menyulitkan pemahaman awal, terutama bagi pengembang baru.

2. Overhead pada Polymorphism
   Penggunaan virtual function dalam jumlah banyak dapat menambah sedikit overhead runtime, meskipun tidak signifikan untuk skala aplikasi ini.

3. Ketergantungan pada Kelas Game
   Beberapa komponen seperti Card dan Tile masih bergantung pada Game untuk mengakses state global sehingga berpotensi meningkatkan coupling jika tidak dikelola dengan baik.

4. Desain yang Relatif Verbose
   Penggunaan banyak kelas kecil dapat meningkatkan jumlah file dan kode yang bisa terasa berlebihan jika tidak dikelola dengan baik.

3.5. Kendala dalam Perancangan

Selama proses perancangan, terdapat beberapa kendala utama:

1. Menjaga Distribusi Tanggung Jawab
   Salah satu tantangan terbesar adalah memastikan bahwa tidak ada kelas yang mengambil terlalu banyak tanggung jawab (menghindari God Class), khususnya pada kelas Game.

2. Desain Sistem Kartu yang Fleksibel
   Banyaknya variasi kartu (ActionCard dan SkillCard) membutuhkan desain yang cukup general, tetapi tetap mampu menangani perilaku spesifik masing-masing kartu.

3. Penyesuaian dengan Aturan Permainan
   Beberapa mekanisme seperti lelang, pajak, dan efek kartu memiliki edge case yang kompleks sehingga memerlukan perhatian khusus dalam desain.

4. Integrasi GUI dan Logic
   Harus dipastikan bahwa GUI tidak mengandung logic permainan sehingga diperlukan pemisahan yang jelas antara kedua layer tersebut.

5. Penanganan Error (Exception Handling)
   Mendesain hierarki exception yang cukup umum, tetapi tetap informatif menjadi tantangan tersendiri.Contohnya kondisi lelang tanpa bid, kebangkrutan, dan efek kartu kompleks (DemolitionCard, FreedomCard).

6. Generalisasi Tanpa Overengineering
   Menjaga keseimbangan antara desain yang fleksibel dan tidak terlalu kompleks.

3.6. Kesimpulan

Secara keseluruhan, desain yang diusulkan telah memenuhi prinsip-prinsip OOP dengan baik serta sesuai dengan spesifikasi dan batasan yang diberikan. Pembagian tanggung jawab yang jelas, penggunaan konsep inheritance dan polymorphism, serta pemisahan antara logic dan GUI menghasilkan sistem yang modular, fleksibel, dan mudah dikembangkan. Meskipun terdapat beberapa kekurangan, seperti kompleksitas yang relatif tinggi, desain ini tetap memberikan keseimbangan yang baik antara fleksibilitas, keterbacaan, dan maintainability sehingga layak digunakan sebagai fondasi dalam pengembangan permainan Nimonspoli.

4. Penerapan Konsep OOP

4.1. Inheritance & Polymorphism

1. Kelas Tile sebagai abstract base class

Semua petak di papan (properti, pajak, festival, penjara, dll) mewarisi dari Tile. Kelas Tile mendeklarasikan metode virtual murni onLanded(), getDisplayInfo(), dan getTileType(). Konsep yang digunakan: Pewarisan kelas dan fungsi virtual (polimorfisme) untuk merepresentasikan berbagai jenis objek yang memiliki perilaku serupa namun berbeda implementasi.

Implementasi dalam kode:

```
class Tile {
public:
    int index;
    std::string code;
    std::string name;
    std::string category;

    Tile();
    Tile(int index, const std::string& code, const std::string& name,
    virtual ~Tile() = default;

    std::string &getName() { return name; }

    virtual void onLanded(Player* player, GameSession* game) = 0;
    virtual void getDisplayInfo(std::stringstream& output) const = 0;
    virtual void getTileType(std::stringstream& output) const = 0;
};
```

Turunannya seperti PropertyTile, TaxTile, FestivalTile, StartTile, JailTile, GoToJailTile, FreeParkingTile, CardTile masing-masing mengimplementasikan metode tersebut sesuai mekanisme spesifik petak.

Keuntungan:

GameSession dapat memanggil tile->onLanded(player, this) tanpa perlu tahu jenis petak sebenarnya.

Menambah jenis petak baru cukup membuat subclass baru tanpa mengubah kode existing.

Memenuhi prinsip Open/Closed dan Liskov Substitution.

2. **Kelas Card dan turunannya (ActionCard, SkillCard)**

   Semua kartu (aksi & kemampuan) memiliki metode execute() (atau use() untuk skill) yang dioverride.

```
class Card {
public:
    std::string id;
    std::string cardType;

    Card(std::string id, std::string type);
    virtual ~Card() = default;
    virtual void execute(Player *player, GameSession *game) = 0;
};
```

Keuntungan:

Mekanisme draw & execute kartu menjadi seragam, terlepas dari efek spesifik kartu.

Memudahkan perluasan jenis kartu di masa depan.

3. Kelas Property dan turunan (StreetProperty, RailroadProperty, UtilityProperty)

   Metode virtual calculateRent() diimplementasikan berbeda sesuai aturan sewa masing-masing jenis properti.

```
class Property {
protected:
    int ID;
    string code;
    string name;
    string type;
    int price;
    int mortgageValue;
    int ownerID;
    raylib::Color colorGroup;
    PropertyStatus status;
    int festivalMultiplier;
    int festivalDuration;

    virtual int calculateRent(int diceNum, int ownedCountOfType, bool isMono) = 0;
```

```
class StreetProperty : public Property {
public:
    int housePrice;
    int hotelPrice;
    vector<int> rentLevels;
    int buildingCount; // 0-4 : rumah, 5 : hotel

    StreetProperty(int ID, string code, string name, int price, int mortgageValue, raylib::Color color, int housePrice, int hotelPrice, vector<int> rentLevels);
    ~StreetProperty();

    int mortgage() override;

    int calculateRent(int diceNum, int ownedCountOfType, bool isMono) override;
    int sellBuilding();
```

4.2. Method/Operator Overloading

Konsep yang digunakan: Overloading operator untuk membuat sintaks lebih alami dan intuitif, terutama pada operasi numerik dan perbandingan.

Implementasi dalam kode:

1. **Operator += dan -= pada kelas Player**

Digunakan untuk menambah/mengurangi uang pemain.

```
Player& Player::operator+=(int amount) {
    addMoney(amount);
    return *this;
}

Player& Player::operator-=(int amount) {
    deductMoney(amount);
    return *this;
}
```

Keuntungan:

a. Memungkinkan penulisan kode seperti player += 500; yang lebih mudah dibaca daripada player.addMoney(500);.
b. Konsisten dengan tipe numerik bawaan.

2. Operator < dan > pada kelas Player

Digunakan untuk membandingkan total kekayaan dua pemain.

```
bool Player::operator<(Player& other) {
    return getTotalWealth() < other.getTotalWealth();
}

bool Player::operator>(Player& other) {
    return getTotalWealth() > other.getTotalWealth();
}
```

Keuntungan:

a. Memudahkan pengurutan pemain berdasarkan kekayaan (misal untuk menentukan pemenang).
b. Sintaks if (player1 > player2) lebih ekspresif dibanding if (player1.getTotalWealth() > player2.getTotalWealth()).

4.3. Template & Generic Classes

Konsep yang digunakan: Template kelas untuk membuat struktur data generik yang dapat menangani berbagai tipe objek.

Implementasi dalam kode:

Kelas CardDeck<T> pada Card.hpp dan Card.tpp – sebuah template untuk tumpukan kartu yang dapat digunakan untuk ActionCard, SkillCard, atau tipe kartu lainnya.

```cpp
template <typename T>
class CardDeck {
private:
    std::vector<T*> activePile;
    std::vector<T*> discardedPile;

public:
    CardDeck() = default;
    ~CardDeck();

    T* draw();
    void discard(T* card);
    void shuffle();
    int size() const;
    bool isEmpty() const;
    void addCard(T* card);

};
```

Keuntungan:

a. Menghindari duplikasi kode untuk deck kartu aksi dan deck kartu skill.
b. Jenis kartu yang berbeda dapat menggunakan logika shuffle/draw/discard yang sama.
c. Memudahkan penambahan jenis deck baru di masa depan (misal deck kartu festival).

4.4. Exception

Konsep yang digunakan: Hierarki exception custom untuk menangani error spesifik aplikasi.

Implementasi dalam kode:

Kelas AppException sebagai base, dengan turunan GameException, PlayerActionException, ResourceException, GraphicsException.

```
class AppException : public std::exception {
public:
    virtual ~AppException() = default;

    virtual const char* what() const noexcept override = 0;
};

class GameException : public AppException {
private:
    std::string message;

public:
    explicit GameException(const std::string& gameComponent, const std
    const char* what() const noexcept override;
};
```

Keuntungan:

a. Memisahkan error game (logika) dari error teknis lainnya.
b. Penanganan exception dapat dilakukan per kategori (misal ResourceException untuk file konfigurasi).
c. Memberikan informasi error yang lebih terstruktur dibanding std::runtime_error biasa.

4.5. C++ Standard Template Library

Konsep yang digunakan: Pemanfaatan container, algoritma, dan utilitas STL secara signifikan.

Implementasi dalam kode:

STL Component	Penggunaan dalam Kode	Alasan
std::vector	Menyimpan daftar pemain (vector<Player*>), properti (vector<Property*>), kartu di deck (vector<T*>), log entry (vector<LogEntry>), dll.	Ukuran dinamis, akses acak O(1), dan mudah diiterasi.
std::map / std::unordered_map	Pemetaan kode petak ke Tile* (tileMap), warna string ke raylib::Color (colorMap), tabel sewa railroad/utility (railroadRentTable, utilityMultiplierTable).	Pencarian berdasarkan key (kode petak, nama warna) dengan cepat.
std::string & std::stringstream	Manipulasi string untuk output, parsing command, format log.	Memudahkan pembuatan pesan dinamis.
std::algorithm	std::shuffle untuk mengacak urutan giliran pemain dan deck kartu, std::find, std::count_if, std::remove.	Menghindari penulisan manual loop untuk operasi umum.
std::random_device, std::mt19937	std::randPengacakan dadu, shuffling deck, random awal.om_device, std::mt19937	Menghasilkan angka acak berkualitas untuk gameplay.

Pengacakan giliran pemain:

```
std::random_device rd; std::mt19937 g(rd());
std::shuffle(players.begin(), players.end(), g);
```

Keuntungan:

a. Mengurangi bug karena algoritma STL sudah teruji.

b. Kode lebih ringkas, mudah dibaca, dan efisien.
c. Memanfaatkan memori yang dikelola secara otomatis oleh container.

4.6. Konsep OOP lain

Selain lima konsep wajib di atas, kelompok penulis juga menerapkan:

1. **Abstract Base Class (Pure Virtual)**

Kelas Tile dan Card memiliki metode pure virtual (=0), sehingga tidak dapat diinstansiasi secara langsung. Ini memaksa setiap turunan untuk mengimplementasikan perilaku spesifik.

```
virtual void onLanded(Player* player, GameSession* game) = 0;
```

2. **Komposisi (Composition)**

   GameSession memiliki objek-objek yang merupakan bagian integral dari permainan, seperti Board*, Dice, TransactionLogger*. Hubungan ini bersifat has-a dan lifecycle dikelola bersama.

```
class GameSession {
private:
    std::vector<Player *> players;
    int currentPlayerIndex;
    int currentTurn;
    int maxTurn;
    int startingBalance;
    Dice dices;
    TransactionLogger *logger;
    GameConfig *config;
    bool isRunning;

    CardDeck<Card> deck;
```

3. **Virtual Destructor**

Semua base class (Tile, Card, Property, AppException) mendeklarasikan destructor virtual untuk memastikan destructor turunan dipanggil saat objek dihapus melalui pointer base.

```
virtual ~Tile() = default;
```

```
virtual ~Card() = default;
```

```
virtual ~Property() = default;
```

4. **Polymorphic Downcasting (dynamic_cast)**

   Beberapa tempat menggunakan dynamic_cast untuk mengakses method spesifik turunan, misal di FestivalTile::onLanded untuk memfilter hanya StreetProperty.

```
std::vector<StreetProperty*> eligibleProperties;
for (Property* ownedProperty : player->getAllProperties()) {
    StreetProperty* street = dynamic_cast<StreetProperty*>(ownedProperty);
    if (street != nullptr) {
        eligibleProperties.push_back(street);
    }
}
```

Meskipun penggunaan dynamic_cast perlu dibatasi, dalam kasus ini diperlukan karena hanya StreetProperty yang memiliki konsep bangunan dan festival mempengaruhi sewa.

Keuntungan keseluruhan:

Penerapan konsep-konsep OOP ini membuat kode lebih modular, mudah diuji (unit test), dan memudahkan perluasan fitur (misal menambah jenis kartu atau petak baru) tanpa mengganggu modul lain.

5. Bonus Yang dikerjakan

5.1. Bonus yang diusulkan oleh spek

5.1.1. GUI

Status: DIKERJAKAN (menggunakan Raylib)

Deskripsi Implementasi:
Kelompok penulis mengimplementasikan antarmuka pengguna grafis (GUI) menggunakan Raylib (raylib-cpp wrapper). GUI ditambahkan sebagai overlay di atas logika permainan yang sudah ada, tanpa mengubah game state atau aturan permainan. Seluruh fungsionalitas CLI tetap berjalan, tetapi pemain dapat memilih untuk bermain melalui tampilan grafis.

Fitur GUI yang diimplementasikan:

- Papan permainan visual: 40 petak digambar dengan warna sesuai kategori (coklat, biru muda, pink, orange, merah, kuning, hijau, biru tua). Setiap petak menampilkan kode properti, nama singkat, dan simbol pemain yang berada di atasnya.
- Bidak pemain: Setiap pemain direpresentasikan dengan lingkaran berwarna berbeda (atau ikon) yang posisinya diperbarui secara dinamis sesuai dengan state terbaru.
- Status properti: Pada petak properti, ditampilkan indikator kepemilikan (warna milik pemain, atau abu-abu jika BANK), status gadai (simbol M), serta jumlah rumah/hotel (gambar kecil atau angka).
- Panel informasi pemain: Menampilkan daftar pemain, uang tunai, jumlah properti, dan posisi saat ini. Panel juga menampilkan kartu kemampuan yang dimiliki pemain aktif.
- Kontrol permainan: Tombol-tombol untuk menjalankan perintah utama (Lempar Dadu, Cetak Papan, Gadai, Tebus, Bangun, Gunakan Kemampuan, Simpan, Keluar). Input manual seperti ATUR_DADU tetap tersedia melalui text input field.
- Log transaksi: Jendela scrollable yang menampilkan log permainan (mirip dengan CETAK_LOG).
- Popup dialog: Untuk konfirmasi pembelian properti, pilihan pajak, pemilihan properti festival, serta input dalam lelang.

Implementasi teknis:

- GUI dipisahkan dalam kelas GameWindow yang berada di layer UI, berkomunikasi dengan GameSession melalui getter dan setter yang sudah ada. Tidak ada logika permainan di dalam GUI.

- Setiap perubahan state (misal setelah LEMPAR_DADU) memicu refresh tampilan dengan membaca data dari GameSession.
- Menggunakan raylib-cpp untuk memudahkan pengelolaan warna, font, dan input.

Keuntungan:

Pemain dapat menikmati visualisasi papan yang lebih intuitif, memudahkan pengambilan keputusan strategis, serta meningkatkan pengalaman bermain. GUI juga membantu pemain baru memahami alur permainan lebih cepat.

5.1.2. COM

Status: DIKERJAKAN

Deskripsi Implementasi:

Kelompok penulis mengimplementasikan Computer Player (COM) yang dapat bertindak sebagai lawan bermain secara otomatis. COM diimplementasikan sebagai turunan dari kelas Player yaitu BotPlayer. Setiap giliran COM, semua aksi dijalankan tanpa campur tangan pemain manusia, dengan output informasi yang jelas (baik di CLI maupun di GUI).

Kemampuan COM:

- Lempar dadu secara otomatis (atau menggunakan ATUR_DADU jika mode debug diaktifkan).
- Keputusan membeli properti: COM membeli properti jika uangnya mencukupi dan properti tersebut belum dimiliki; untuk Railroad dan Utility, COM langsung mengklaim sesuai aturan.
- Menawar lelang: COM mengikuti lelang dengan strategi sederhana: menawar hingga batas tertentu (misal ≤ 70% uang tunai) atau pass jika harga melebihi ambang batas.
- ‘Pajak: COM membandingkan opsi flat vs persentase dan memilih yang lebih murah (dengan menghitung total kekayaan terlebih dahulu).
- Festival: COM memilih properti dengan potensi sewa tertinggi (atau properti yang sering didatangi lawan) untuk digandakan.
- Gadai & Tebus: COM menggadaikan properti yang tidak produktif saat butuh uang, dan menebus properti (terutama yang berada dalam color group yang hampir monopoli) saat uang cukup.
- Bangunan: COM membangun rumah/hotel secara merata pada color group yang sudah dimonopoli, prioritas pada properti dengan nilai sewa tinggi.

- Kartu kemampuan: COM menggunakan kartu pada momen yang tepat (misal ShieldCard saat uang hampir habis, TeleportCard untuk mendarat di properti yang bisa dibeli atau yang memberikan keuntungan, DemolitionCard untuk menghancurkan bangunan lawan yang paling menguntungkan).
- Keluar dari penjara: COM membandingkan opsi bayar vs lempar dadu (memilih bayar jika probabilitas double < 1/3 atau jika uang cukup longgar).
- Bankruptcy: COM melakukan likuidasi aset secara otomatis (menjual bangunan, menggadaikan properti) sesuai urutan yang memaksimalkan kas.

Implementasi teknis:

- BotPlayer::chooseInput(const std::vector<int>& choices) tidak hanya memilih opsi pertama, tetapi menjalankan evaluasi internal (berdasarkan kondisi GameSession yang diberikan melalui parameter tambahan). Untuk itu, kami menambahkan GameSession* sebagai argumen atau menyimpan pointer ke GameSession di dalam BotPlayer.
- Semua aksi COM dicatat di TransactionLogger sehingga pemain dapat melihat riwayat keputusan COM.
- COM dapat diaktifkan dengan memilih tipe pemain "Bot" saat New Game (selain Human). Jumlah maksimal COM adalah 3 (karena total pemain maksimal 4).

Keuntungan:

Pemain tetap dapat menikmati permainan meskipun tidak memiliki teman (single player vs COM). COM menjadi lawan latih yang cukup menantang dan membantu menguji strategi.

5.2. Bonus Kreasi Mandiri

<u>Kartu Kemampuan Tambahan: FreedomCard (Bebas dari Penjara)</u>

Deskripsi Singkat:

Kelompok penulis mengimplementasikan satu jenis kartu kemampuan tambahan di luar daftar kartu yang secara eksplisit didefinisikan dalam spesifikasi. Kartu ini bernama FreedomCard (Kartu Bebas dari Penjara). Di dalam dokumen spesifikasi, kartu "Bebas dari Penjara" hanya disebut sekilas sebagai salah satu opsi keluar penjara, tanpa diberikan mekanisme detail maupun dimasukkan ke dalam daftar kartu kemampuan spesial yang wajib diimplementasikan. Oleh karena itu, kami memutuskan untuk merancang dan mengimplementasikan kartu ini sebagai fitur tambahan (bonus kreativitas).

Mekanisme Kartu:

Efek: Ketika digunakan, FreedomCard akan mengubah status pemain dari JAILED menjadi ACTIVE dan mereset hitungan percobaan keluar penjara (jailAttempts) menjadi 0.

Cara Mendapatkan: Kartu ini dapat diambil dari tumpukan kartu kemampuan (SkillCard) saat pemain mendapatkan kartu acak di awal giliran (sesuai aturan pembagian kartu kemampuan spesial).

Batasan Penggunaan: Kartu hanya dapat digunakan ketika pemain sedang berada dalam status penjara (PlayerStatus::JAILED). Jika digunakan di luar kondisi tersebut, kartu tidak memberikan efek dan tetap berada di tangan pemain (atau dapat dirancang untuk tidak dapat dipilih).

Sekali Pakai: Setelah digunakan, kartu akan dibuang (discarded) dari tangan pemain dan masuk ke tumpukan discard seperti kartu kemampuan lainnya.

Prioritas Penggunaan: Kartu ini menjadi salah satu opsi yang ditawarkan kepada pemain ketika berada dalam penjara, bersama dengan opsi membayar denda atau mencoba melempar dadu double. Di GUI, opsi ini ditampilkan sebagai tombol "Gunakan Kartu Bebas Penjara" jika pemain memiliki kartu tersebut.

Implementasi Teknis:

Kartu ini diimplementasikan sebagai turunan dari kelas SkillCard:

```
class FreedomCard : public SkillCard {
public:
    FreedomCard(std::string id);
    void use(Player *player, GameSession *game) override;
};
```

6. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
Papan Permainan (Board)	13524060	13524004, 13524095
Petak Properti, Petak Aksi, Petak Spesial (Tiles)	13524060	13524004, 13524095
Dadu (Dice)	13524004	13524004, 13524095
Properti (Property, StreetProperty, RailroadProperty, UtilityProperty)	13524088	13524004, 13524095
Sewa (Rent)	13524060	13524004, 13524095
Gadai (Mortgaged)	13524088	13524004, 13524095
Peningkatan Properti (Build)	13524088	13524004, 13524095
Lelang Tanah (Auction)	13524088	13524004, 13524095
Pajak	13524060	13524004, 13524095
Festival	13524060	13524004, 13524095
Kartu Kesempatan dan Dana Umum	13524004, 13524060	13524004, 13524095
Kartu Kemampuan Spesial	13524004, 13524060	13524004, 13524095
Bangkrut	13524088	13524004, 13524095
Pengambilalihan Asset	13524088	13524004, 13524095

Kondisi Game Selesai, dan Mulai	13524004	13524004, 13524095
Save/Load	13524071	13524004, 13524095
Transaction Logger	13524071	13524004, 13524095

Lampiran

Lampiran A. Form Asistensi

IF2010_TB1_Asistensi_SSR.pdf